



C#_theme-12 NuGet та JSON

У цій темі представлено огляд NuGet, менеджера пакетів для платформи .NET, а також використання JSON для обміну даними. Ми розглянемо, як встановлювати пакети за допомогою NuGet, а також серіалізацію та десеріалізацію даних у форматі JSON за допомогою бібліотеки Newtonsoft.Json.

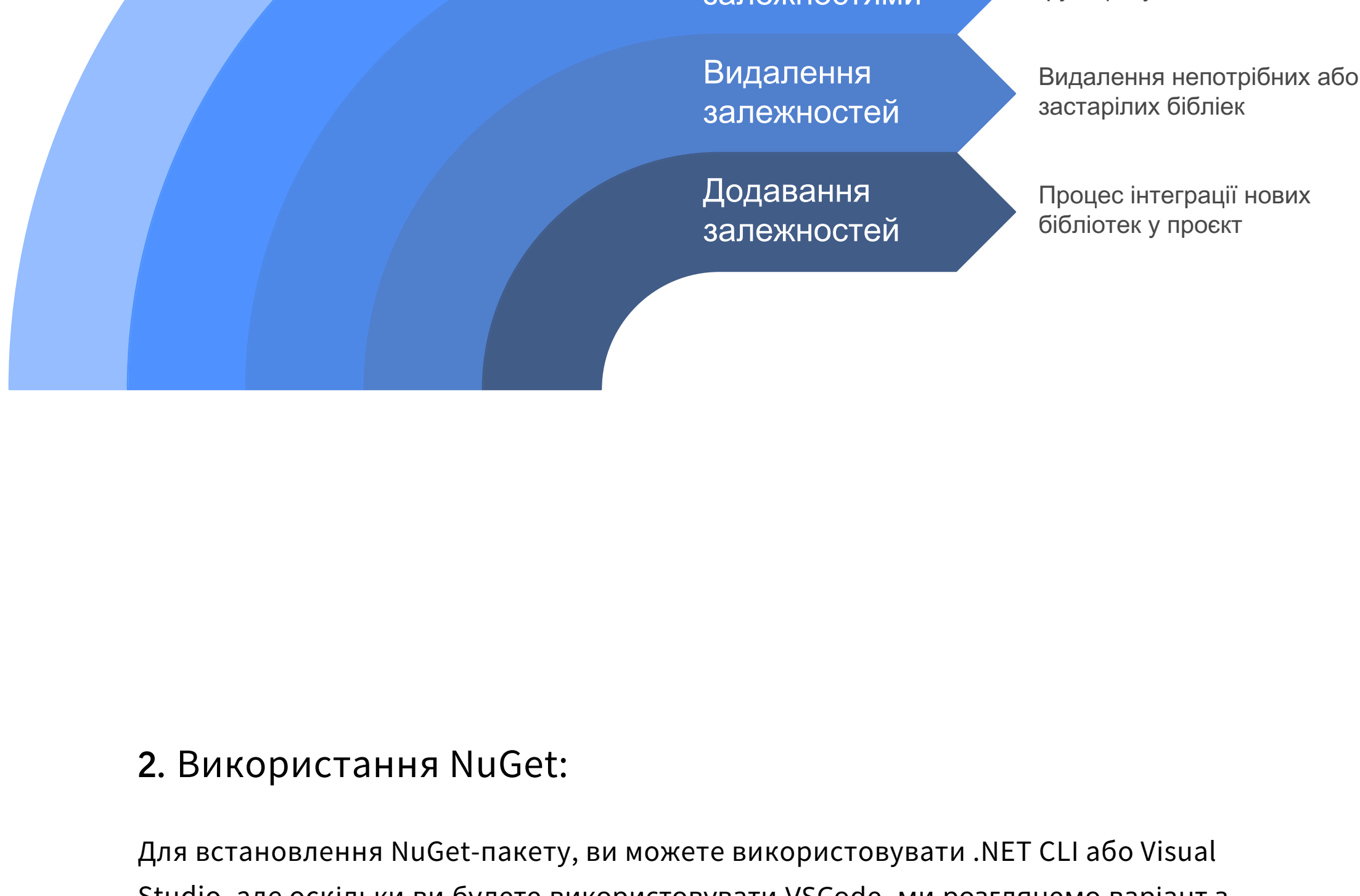


NuGet - це менеджер пакетів для платформи .NET, який дозволяє додавати, видаляти та керувати залежностями вашого проєкту.

1. Що таке NUGET?

NuGet - це менеджер пакетів для платформи .NET, який дозволяє додавати, видаляти та керувати залежностями вашого проєкту. Він спрощує процес встановлення та оновлення бібліотек, які ви використовуєте в своєму проєкті. NuGet дозволяє швидко та зручно включати сторонні бібліотеки в вашу програму.

Розуміння NuGet



2. Використання NuGet:

Для встановлення NuGet-пакету, ви можете використовувати .NET CLI або Visual Studio, але оскільки ви будете використовувати VSCode, ми розглянемо варіант з

Крок 1: Встановлення NuGet-пакету

Для встановлення NuGet-пакету, використовуйте команду:

```
dotnet add package Назва_Пакету
```

Наприклад, для встановлення пакету Newtonsoft.Json:

```
dotnet add package Newtonsoft.Json
```

Крок 2: Використання NuGet-пакету в коді

Після встановлення пакету, ви можете використовувати його в своєму коді. Наприклад, для роботи з Newtonsoft.Json для серіалізації та десеріалізації JSON:

```
using Newtonsoft.Json; // Додайте цей import у вашому коді

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main()
    {
        // Серіалізація об'єкта в JSON
        Person person = new Person { Name = "John", Age = 30 };
        string json = JsonConvert.SerializeObject(person);

        // Десеріалізація JSON у об'єкт
        Person deserializedPerson = JsonConvert.DeserializeObject<Person>(json);

        Console.WriteLine($"Name: {deserializedPerson.Name}, Age: {deserializedPerson.Age}");
    }
}
```

Dapper, FluentValidation, Serilog

- **Dapper:** Dapper - це легка ORM для роботи з базами даних, яка допоможе спростити роботу з SQL-запитами.
- **FluentValidation:** FluentValidation - це бібліотека для валідації даних у вашому програмному коді.
- **Serilog:** Serilog - це бібліотека для логування подій та помилок у вашій

Ці пакети можуть бути встановлені та використані аналогічно до Newtonsoft.Json за допомогою NuGet.



Що таке JSON?

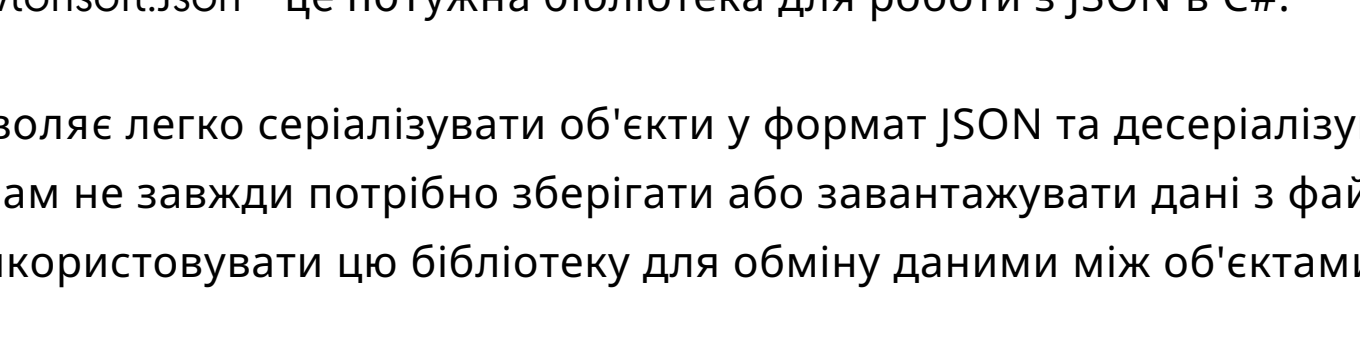
JSON - це компактний формат обміну даними, який був початково розроблений для JavaScript, але зараз використовується широкою гамою програмних мов.

JSON це по суті спосіб збереження та передачі структурованих даних у текстовому вигляді. Ось декілька ключових аспектів JSON і причини, чому ми його потребуємо:

- Простота читання та запису: JSON використовує чітку та легку синтаксичну структуру, що робить його легко зрозумілим для людей. JSON дані подібні до словника з ключами і значеннями, і вони можуть бути легко читані та
- Легкість інтерпретації для машин: JSON також легко інтерпретується та обробляється програмами і мовами програмування. Багато мов підтримують бібліотеки для роботи з JSON, що робить обробку даних у цьому форматі
- Підтримка структурованих даних: JSON дозволяє зберігати структуровані дані, такі як масиви та об'єкти, що дозволяє легко організувати та передавати складні структури даних.
- Популярність у веб-розробці: JSON є стандартним форматом для обміну даними в сучасних веб-застосунках. Багато веб-сервісів та API повертають дані у форматі JSON, і це дозволяє взаємодіяти з ними зручно та ефективно.
- Розширюваність: JSON дозволяє включати додаткові дані або метадані до об'єктів, які надають додаткову інформацію про дані, не порушуючи загальну структуру. Це дозволяє розширювати та адаптувати дані для конкретних

Загалом, JSON є важливим інструментом для обміну та зберігання даних, особливо в контексті веб-розробки, мікросервісів, обробки даних та багатьох інших областей програмування. Цей формат спрощує передачу та обробку даних, що робить його незамінним для сучасних програмістів.

Використання JSON



Використання Newtonsoft.Json

Використання Newtonsoft.Json для серіалізації та десеріалізації даних.

Newtonsoft.Json - це потужна бібліотека для роботи з JSON в C#.

Вона дозволяє легко серіалізувати об'єкти у формат JSON та десеріалізувати JSON у об'єкти. Вам не завжди потрібно зберігати або завантажувати дані з файлів, можна просто використовувати цю бібліотеку для обміну даними між об'єктами та

Серіалізація в JSON:

Серіалізація - це процес перетворення об'єкта в формат JSON. Ваш об'єкт стає рядком JSON. Ось приклад:

```
using Newtonsoft.Json;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main()
    {
        // Створюємо об'єкт типу Person
        Person person = new Person { Name = "Іван", Age = 25 };

        // Серіалізація в JSON
        string json = JsonConvert.SerializeObject(person);

        // Вивід серіалізованого JSON у консоль
        Console.WriteLine(json);
    }
}
```

У цьому прикладі об'єкт типу Person був серіалізований у рядок JSON.

Десеріалізація з JSON:

Десеріалізація - це процес перетворення JSON назад у об'єкт C#. Ось як це працює:

```
using Newtonsoft.Json;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main()
    {
        // Рядок JSON, який ми хочемо десеріалізувати
        string json = "{\"Name\":\"Марія\",\"Age\":30}";

        // Десеріалізація JSON у об'єкт
        Person deserializedPerson = JsonConvert.DeserializeObject<Person>(json);

        // Вивід десеріалізованого об'єкта
        Console.WriteLine($"Ім'я: {deserializedPerson.Name}, Вік: {deserializedPerson.Age}");
    }
}
```

У цьому прикладі ми використали метод JsonConvert.DeserializeObject, щоб створити об'єкт типу Person з рядка JSON.



Використання File.ReadAllText() та File.WriteAllText() з Newtonsoft.Json для зберігання та завантаження даних.

Для зберігання та завантаження даних з файлу за допомогою Newtonsoft.Json в C#, ви можете використовувати методи File.ReadAllText() та File.WriteAllText() для читання та запису JSON даних у файл. Ось приклад, як це можна зробити:

```
using System;
using System.IO;
using Newtonsoft.Json;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main()
    {
        // Створення об'єкта типу Person
        Person person = new Person { Name = "Іван", Age = 25 };

        // Серіалізація об'єкта в JSON
        string json = JsonConvert.SerializeObject(person);

        // Зберігання JSON у файл
        File.WriteAllText("person.json", json);

        // Завантаження JSON з файлу
        string loadedJson = File.ReadAllText("person.json");

        // Десеріалізація JSON у об'єкт
        Person deserializedPerson = JsonConvert.DeserializeObject<Person>(loadedJson);

        // Вивід десеріалізованого об'єкта
        Console.WriteLine($"Ім'я: {deserializedPerson.Name}, Вік: {deserializedPerson.Age}");
    }
}
```

У цьому прикладі ми:

1. Створюємо об'єкт типу Person.
2. Серіалізуємо цей об'єкт в JSON та зберігаємо його у файл "person.json" за допомогою File.WriteAllText().
3. Завантажуємо JSON з файлу "person.json" за допомогою File.ReadAllText().
4. Десеріалізуємо JSON у об'єкт типу Person.
5. Виводимо десеріалізований об'єкт на консоль.

Процес серіалізації та десеріалізації

